

Using Type Erasure to Extend APIs You Don't Own

A Case Study From Audio Plugin Development

Jan Wilczek (think-cell)

The Dutch C++ Group Meetup

Amsterdam, September 4th, 2026

Who am I?

Who am I?

- Jan Wilczek [Yan Vil-check]

Who am I?

- Jan Wilczek [Yan Vil-check]
- C++ Speaker at think-cell

Who am I?

- Jan Wilczek [Yan Vil-check]
- C++ Speaker at think-cell
- Founder of TheWolfSound.com & online course creator



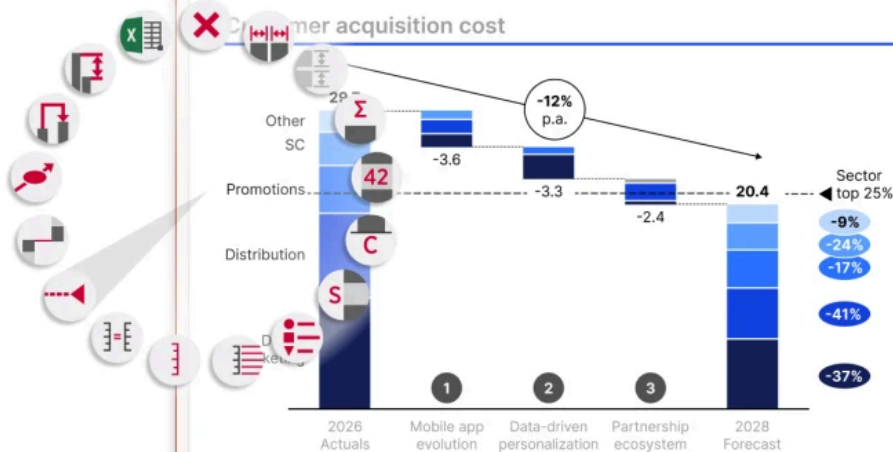
Access 250+ best-practice templates



Use AI-powered tools inside PowerPoint and Excel

Minimize customer acquisition

cost by 12% p.a. to approach sector benchmarks by 2028



Key projects and funding required



Automate charts with Excel-linking

1. Mobile app
• Booking engine
• Self-service purchase, seat selection

	A	B	C	D
1	Category	Digital mark..	Distribution..	Promotions..
2	2026 Actuals (Total)	13	10	5
3	Mobile app evolution	-3.6		
4	Data-driven personal..	-3.3		
5	Partnership ecosyst...	-2.4		
6	2028 Forecast	8	5	3



Create management-level charts and presentations

Who knows what audio plugins for digital audio workstations are?

Who has heard about the JUCE C++ framework?

Untitled

Link Tap 167.00 4 / 4 1 Bar C# / D# Minor 1. 1. 1 108 0. 0 Key MIDI 48.0 kHz 13 %

Search (Cmd + F)

Collections

- Content
- Function
- Format

Library

- All
- Sounds
- Drums
- Grooves
- Instruments
- Audio Effects
- MIDI Tools
- Presets
- Samples
- Templates
- Max for Live
- Plug-Ins
- Clips
- Samples
- Grooves
- Tunings
- Templates

Places

- Packs
- Splice
- Cloud
- Push
- User Library
- Current Project
- Add Folder...

Content

Intro Verse 1 Chorus Verse 2 Chorus Breakdown Instrumental Chorus Outro

Coffee Leaf Coffee Leaf Coffee Leaf

Strum-o-M Strum-o-M Strum-o-M Strum-o-M Strum-o-M Strum-o-M Strum-o-M Strum-o-M Strum-o-M Strum-o-M

Drums Drums Drums Drums Drums Drums Drums Drums

The Beat O The Beat C The Beat C The Beat C The Beat C The Beat C The Beat C The Beat C

Mello

Rose Bass Rose Bass Rose Bass Rose Bass Rose Bass

Sub Sub B Sub Sub B Sub Sub B Sub Sub B

Guit Layte Guit Layte

Transform Transform Transform Transform Transform Transform Transform Transform

Vocals Patience Main Vocals Patience Harmony Vocals Patience

Chuck Sutton - Patience (Live 12 Demo)

Drop Files and Devices Here

Del

Coffee Leaf 1 S

Strum 2 S

Drums 3 S

The Beat Of Histor 4 S

Mello 5 S

Rose Bass 6 S

Sub Sub Bass 7 S

Guit Layter 8 S

Transform Seed 9 S

Vacuum 10 S

Vocal Main 11 S

Vocal Harmony 12 S

Vocal Hum 13 S

Vocal Adlib 14 S

Rainstorm 15 S

A Reverb A S Post

B Delay B S Post

Main 1/2 0 0

Coffee Leaf Strum Drums The Beat O Mello Rose Bass Sub Sub B Guit Layter Transform Se Vacuum Vocal Main Vocal Harmon Vocal Hum Vocal Adlib Rail A Reverb B Delay Main

Sends

1 2 3 4 5 6 7 8 9 10 11 12 13 14 A B Solo

Coffee Leaf

Osc 1 Oct Shape Slide 1 Osc 1 -3.2 dB

Osc 2 Oct Detune 2 Osc 2 -3.2 dB

Pitch Mod Env 2 LFO 0.0 % 0.0 %

Filter

Freq Type 355 Hz Res HP 0.59 10.0 Hz

Envelopes

1 Attack Decay Sustain Release 114 ms 17.2 ms 81 % 544 ms

2 Attack Decay Sustain Release 41.5 ms 5.19 ms 0.0 % 1.25 s

LFO

Rate 0.44 Hz 1:1

Mod

Mode Poly

Voices 32

Drift 0.0 %

Legato

Glide 8.84 ms

Utility

Input Gain -30 dB

Output Gain 0.00 dB

Balance 0

Mute DC

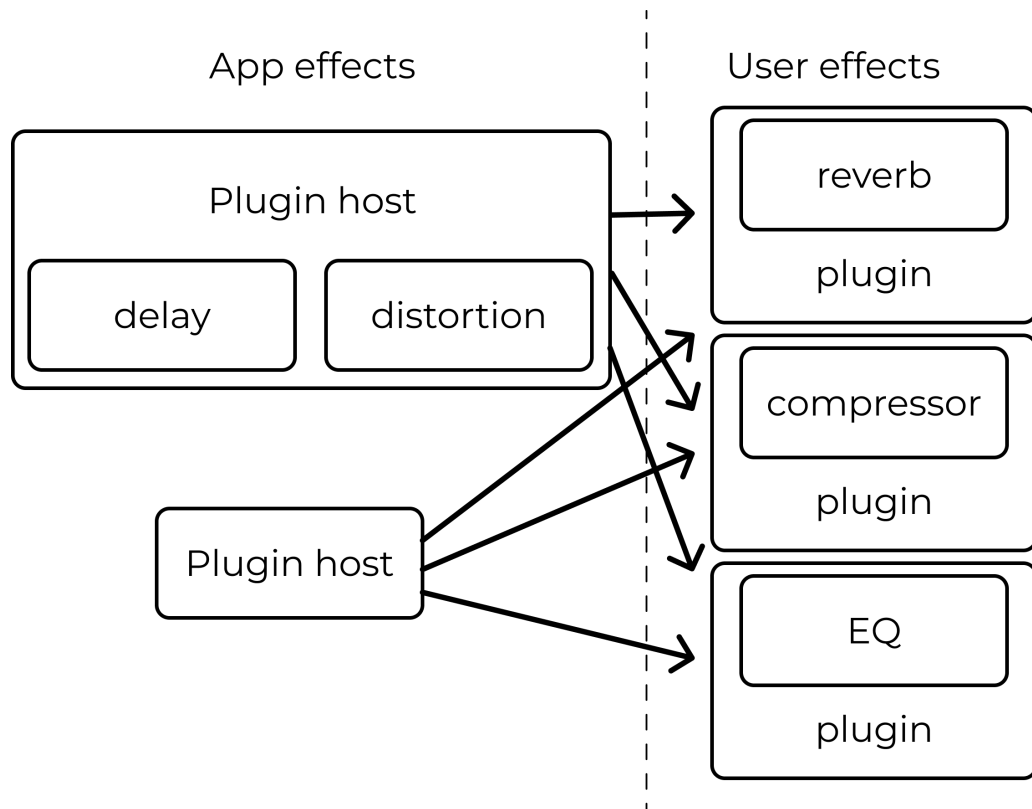
Happy Bees

Swarm

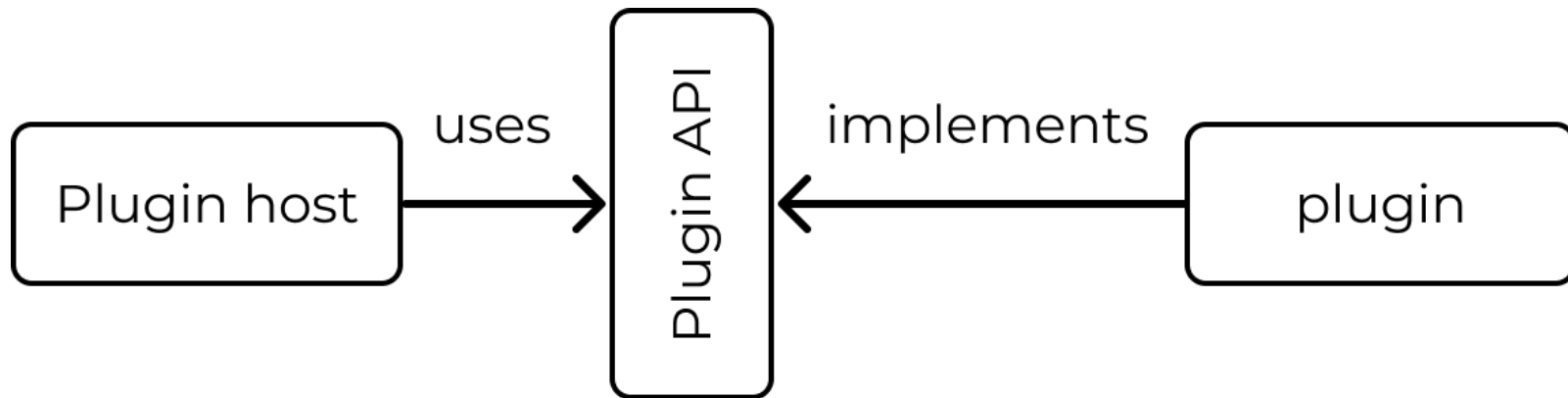
Drop Audio Effects Here

Coffee Leaf

Audio plugins



Audio plugin formats



Audio plugin formats

reverb

VST3 plugin

reverb

AUv3 plugin

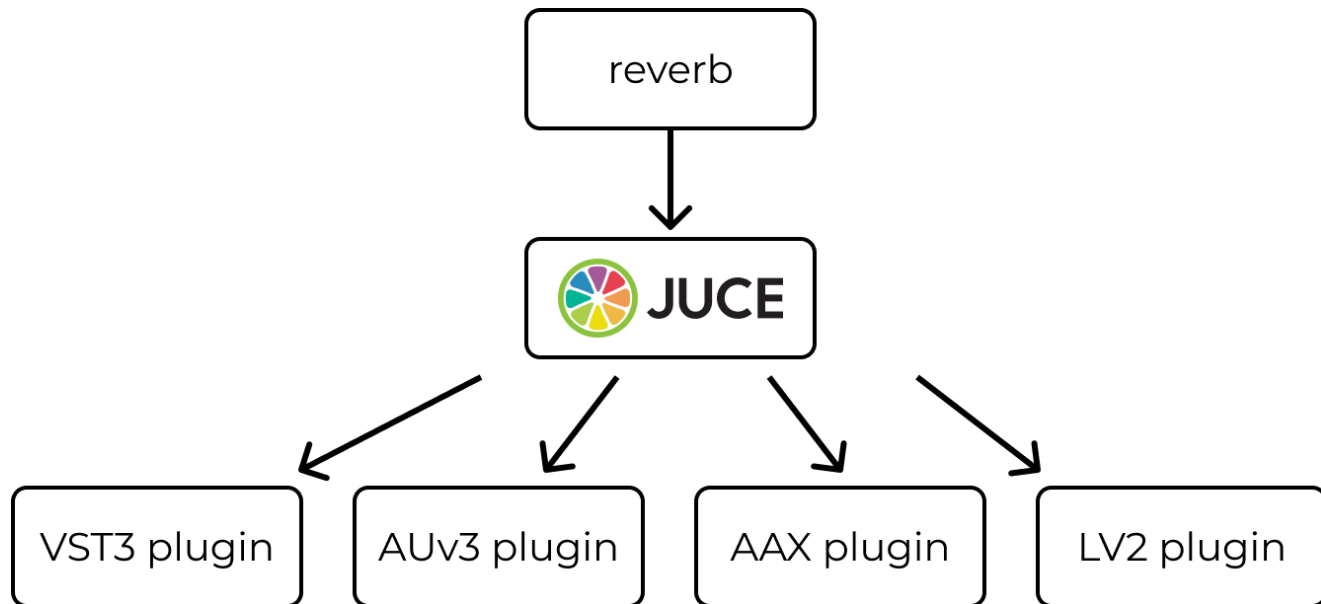
reverb

AAX plugin

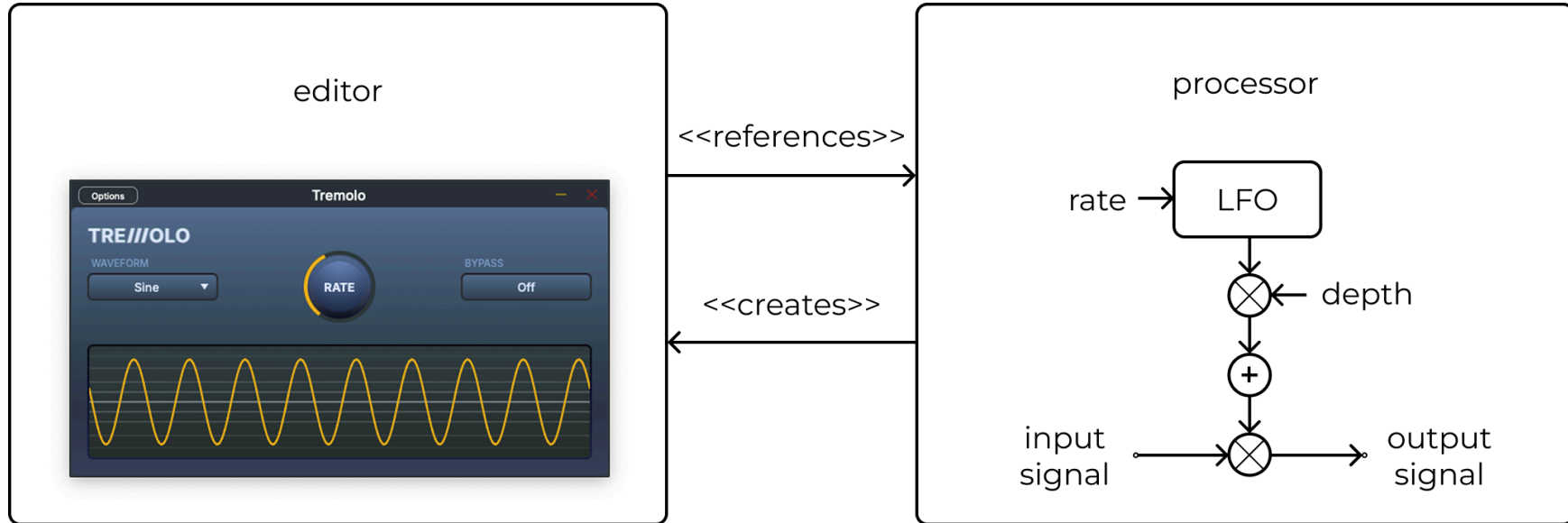
reverb

LV2 plugin

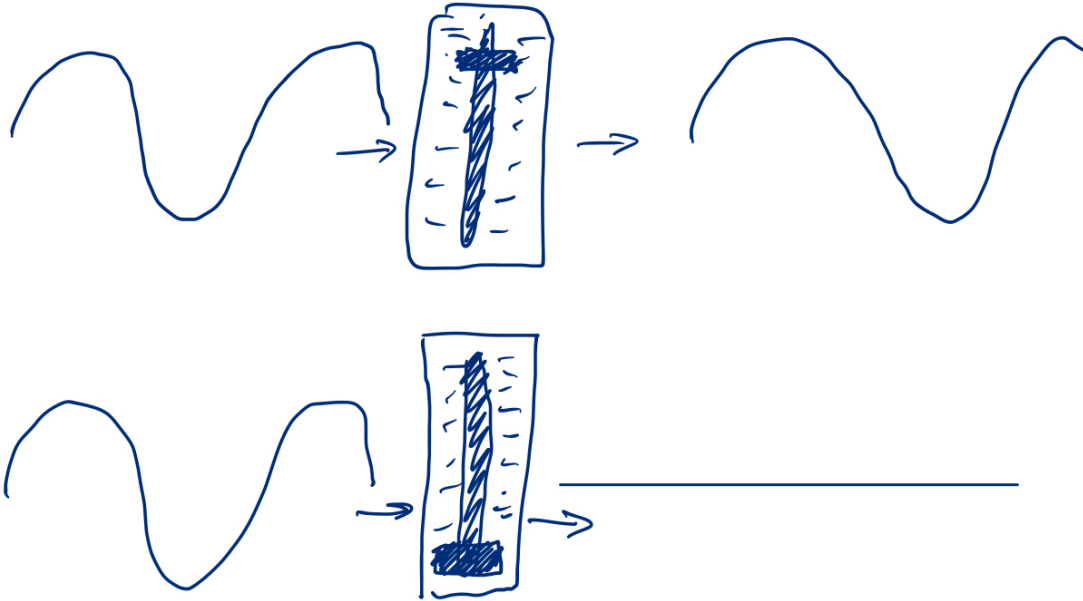
JUCE C++ framework



Plugin processor and editor



Gain parameter example



Gain parameter requirements

Gain parameter requirements

- it controls the volume of the sound output by the plugin

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI
- it must be reported to the host to meet its requirements (requirement of plugin APIs)

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI
- it must be reported to the host to meet its requirements (requirement of plugin APIs)
- it can be adjusted by the user via a slider in the plugin's UI

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI
- it must be reported to the host to meet its requirements (requirement of plugin APIs)
- it can be adjusted by the user via a slider in the plugin's UI
- it can be adjusted by the user in host's automation view

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI
- it must be reported to the host to meet its requirements (requirement of plugin APIs)
- it can be adjusted by the user via a slider in the plugin's UI
- it can be adjusted by the user in host's automation view
- it must be persisted between DAW project reloads

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI
- it must be reported to the host to meet its requirements (requirement of plugin APIs)
- it can be adjusted by the user via a slider in the plugin's UI
- it can be adjusted by the user in host's automation view
- it must be persisted between DAW project reloads
- it should be easy to serialize for user presets

Gain parameter requirements

- it controls the volume of the sound output by the plugin
- it must be displayed in the UI
- it must be reported to the host to meet its requirements (requirement of plugin APIs)
- it can be adjusted by the user via a slider in the plugin's UI
- it can be adjusted by the user in host's automation view
- it must be persisted between DAW project reloads
- it should be easy to serialize for user presets
- it's access must be real-time-safe (read/written on the audio thread, updated on the UI thread)

Recap

Recap

- **audio plugin:** a plugin for a digital audio workstation (DAW)

Recap

- **audio plugin:** a plugin for a digital audio workstation (DAW)
- **plugin processor:** the core of an audio plugin; responsible for plugin metadata, audio processing, and state management

Recap

- **audio plugin:** a plugin for a digital audio workstation (DAW)
- **plugin processor:** the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor:** user interface (UI) of the plugin

Recap

- **audio plugin**: a plugin for a digital audio workstation (DAW)
- **plugin processor**: the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor**: user interface (UI) of the plugin
- **plugin parameter**: a user-controllable value influencing audio processing of a plugin
 - `juce::AudioParameterFloat|Bool|Int|Choice` class instance in JUCE plugins

Recap

- **audio plugin:** a plugin for a digital audio workstation (DAW)
- **plugin processor:** the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor:** user interface (UI) of the plugin
- **plugin parameter:** a user-controllable value influencing audio processing of a plugin
 - `juce::AudioParameterFloat|Bool|Int|Choice` class instance in JUCE plugins
- **UI state:** non-audio-related state

Recap

- **audio plugin**: a plugin for a digital audio workstation (DAW)
- **plugin processor**: the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor**: user interface (UI) of the plugin
- **plugin parameter**: a user-controllable value influencing audio processing of a plugin
 - `juce::AudioParameterFloat|Bool|Int|Choice` class instance in JUCE plugins
- **UI state**: non-audio-related state
- **serialization**: process of externalizing state (for example, to a JSON file)

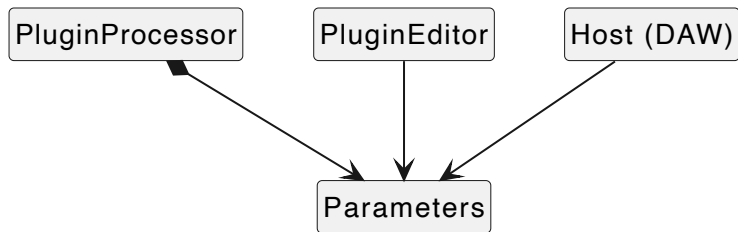
Recap

- **audio plugin:** a plugin for a digital audio workstation (DAW)
- **plugin processor:** the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor:** user interface (UI) of the plugin
- **plugin parameter:** a user-controllable value influencing audio processing of a plugin
 - `juce::AudioParameterFloat|Bool|Int|Choice` class instance in JUCE plugins
- **UI state:** non-audio-related state
- **serialization:** process of externalizing state (for example, to a JSON file)
- **deserialization:** process of loading state from an external source (for example, a JSON file)

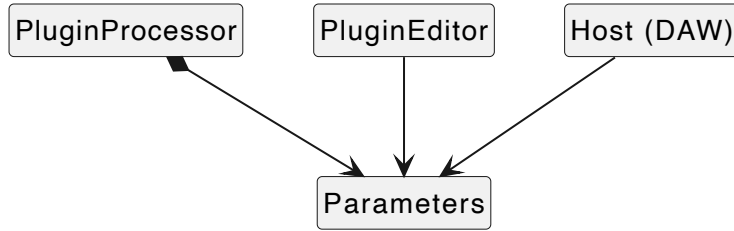
Recap

- **audio plugin**: a plugin for a digital audio workstation (DAW)
- **plugin processor**: the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor**: user interface (UI) of the plugin
- **plugin parameter**: a user-controllable value influencing audio processing of a plugin
 - `juce::AudioParameterFloat|Bool|Int|Choice` class instance in JUCE plugins
- **UI state**: non-audio-related state
- **serialization**: process of externalizing state (for example, to a JSON file)
- **deserialization**: process of loading state from an external source (for example, a JSON file)
- **preset** = plugin parameter values + metadata

Parameters in audio plugins

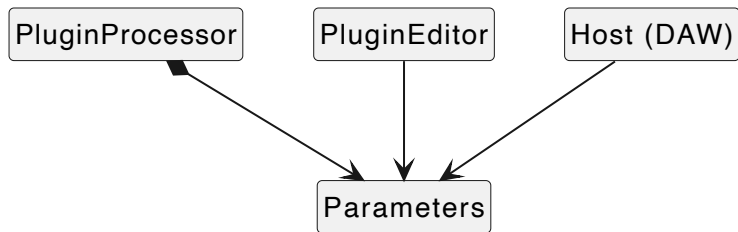


Parameters in audio plugins



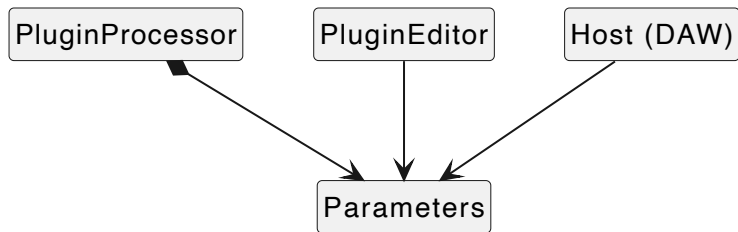
- Automation

Parameters in audio plugins



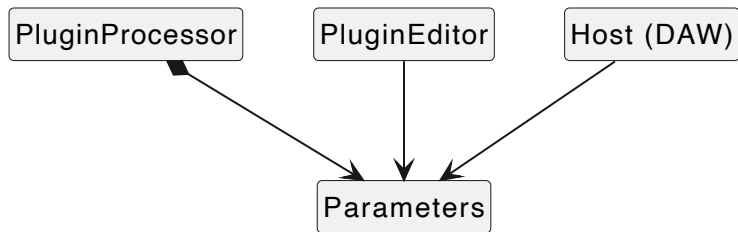
- Automation
- Serialization

Parameters in audio plugins



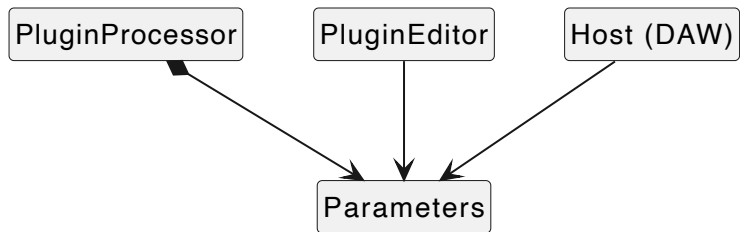
- Automation
- Serialization
- Presets (generic/custom)

Parameters in audio plugins



- Automation
- Serialization
- Presets (generic/custom)
- Visualization (generic/custom)

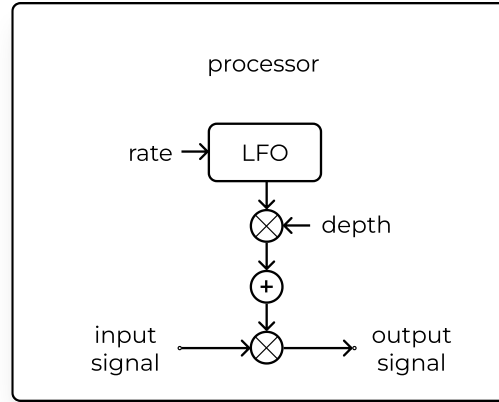
Parameters in audio plugins



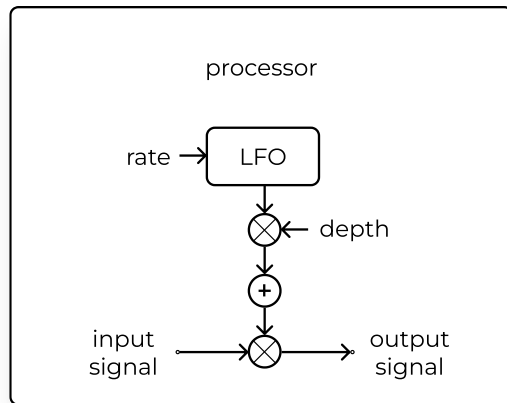
- Automation
- Serialization
- Presets (generic/custom)
- Visualization (generic/custom)
- UI (generic/custom)

Example of plugin parameters in action

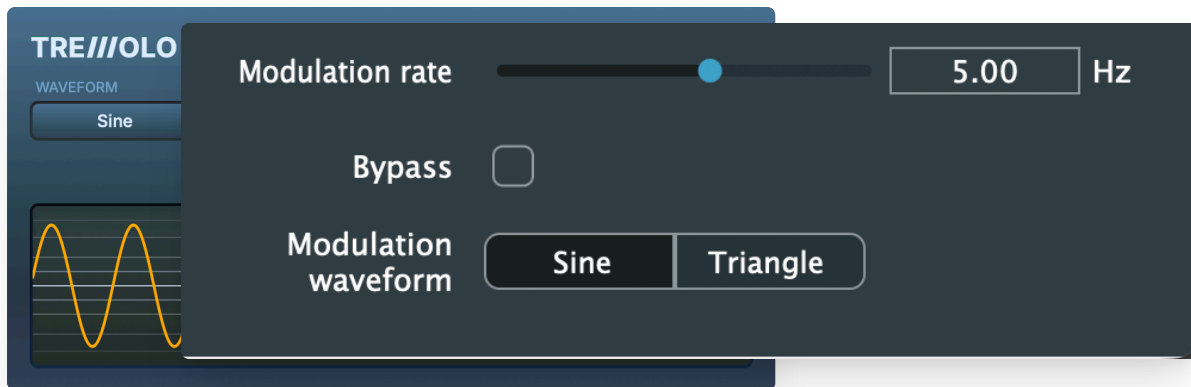
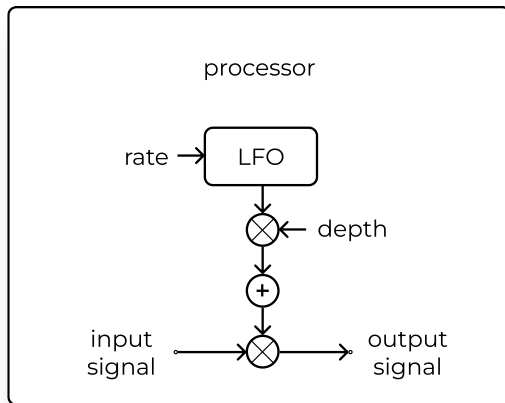
Example of plugin parameters in action



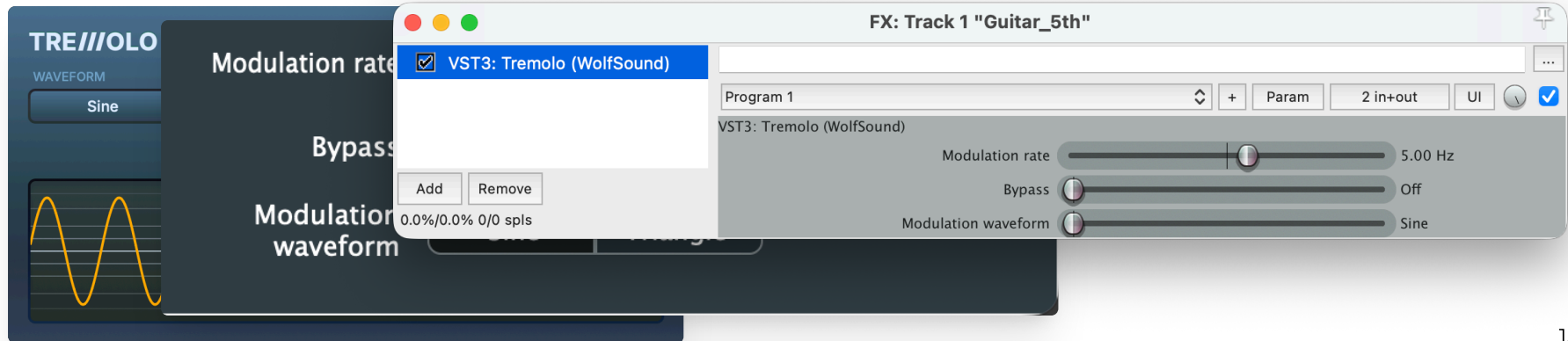
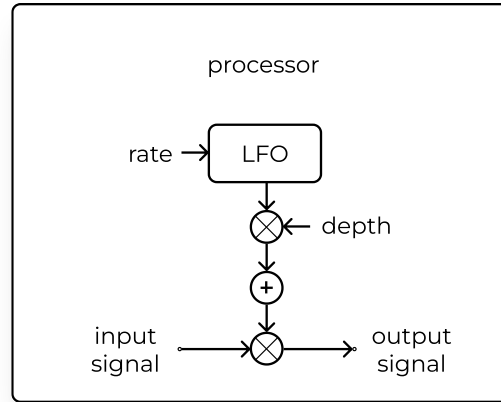
Example of plugin parameters in action



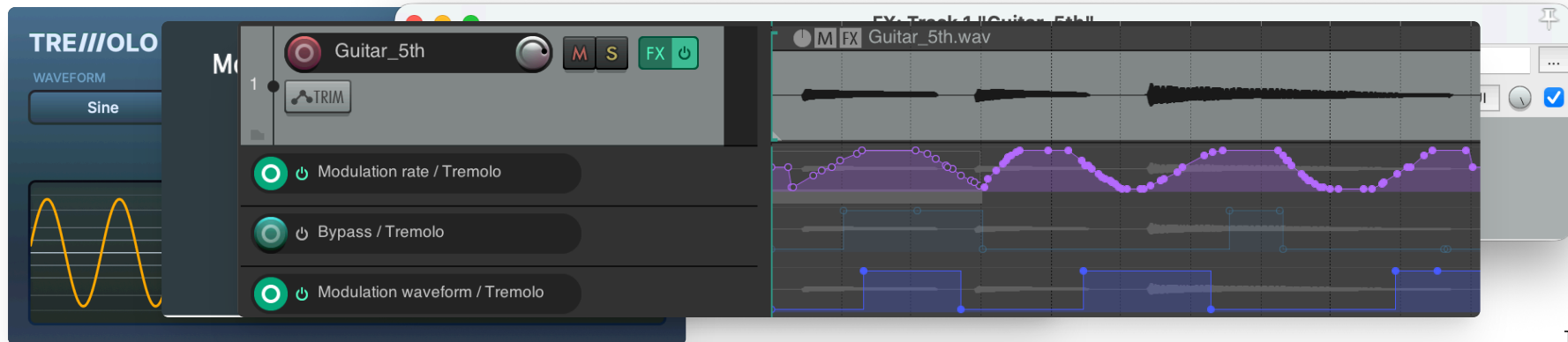
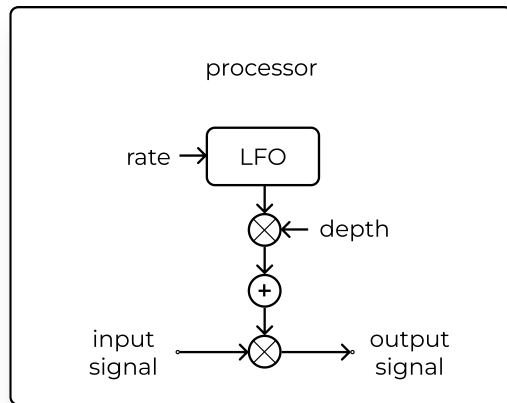
Example of plugin parameters in action



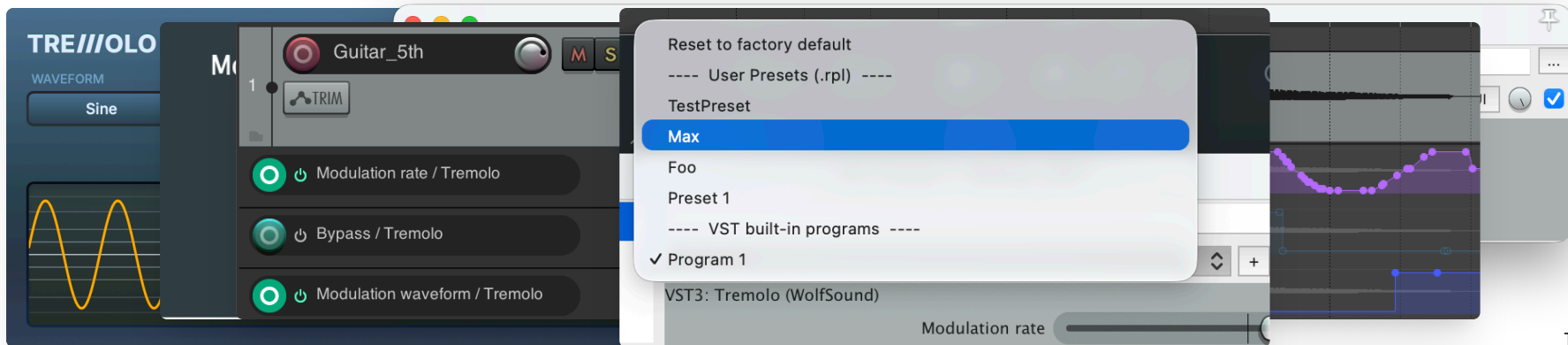
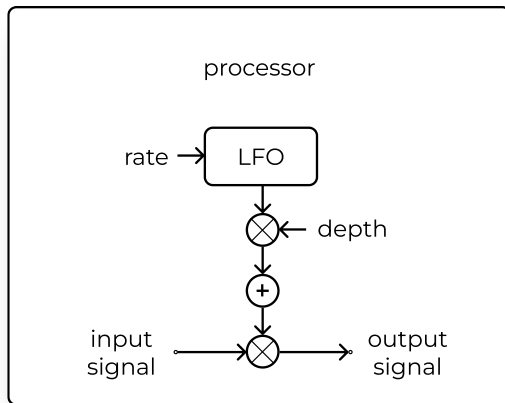
Example of plugin parameters in action



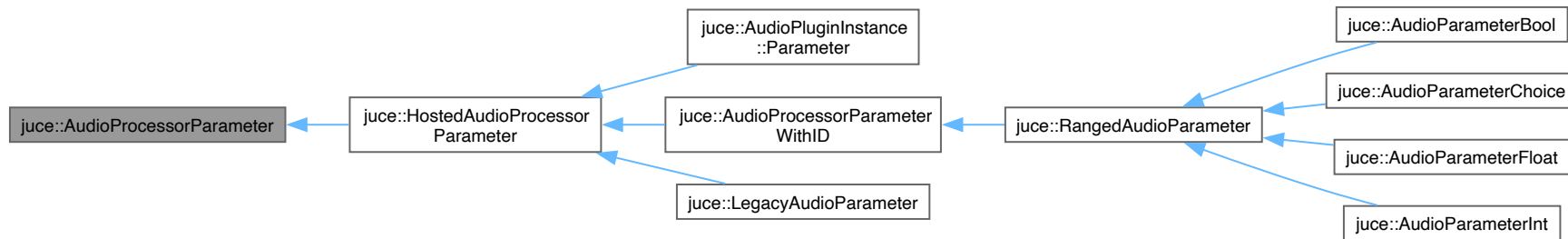
Example of plugin parameters in action



Example of plugin parameters in action



Parameter class hierarchy in JUCE



JUCE parameter classes

JUCE parameter classes

- `AudioParameterBool`
 - `true / false`

JUCE parameter classes

- `AudioParameterBool`
 - `true / false`
- `AudioParameterInt`
 - integer in a closed range
 - e.g., "1" from {0, 1, 2}

JUCE parameter classes

- `AudioParameterBool`
 - `true / false`
- `AudioParameterInt`
 - integer in a closed range
 - e.g., "1" from {0, 1, 2}
- `AudioParameterFloat`
 - real value from a closed range
 - e.g., "0.25" from [0, 2]

JUCE parameter classes

- `AudioParameterBool`
 - `true / false`
- `AudioParameterInt`
 - integer in a closed range
 - e.g., "1" from {0, 1, 2}
- `AudioParameterFloat`
 - real value from a closed range
 - e.g., "0.25" from [0, 2]
- `AudioParameterChoice`
 - a value from a fixed set of named options
 - e.g., "lowpass" from {"lowpass", "highpass"}

How do we use parameters?

Plugin processor

```
1  class PluginProcessor : public AudioProcessor {
2  public:
3      PluginProcessor();
4      ~PluginProcessor();
5
6      void prepareToPlay(double sampleRate, int samplesPerBlock) override;
7      void releaseResources() override;
8
9      #ifndef JUCEPlugin_PreferredChannelConfigurations
10     bool isBusesLayoutSupported(const BusesLayout& layouts) const override;
11     #endif
12
13     void processBlock(AudioBuffer<float>&, MidiBuffer&) override;
14
15     AudioProcessorEditor* createEditor() override;
16     bool hasEditor() const override;
17
18     const String getName() const override;
19
20     bool acceptsMidi() const override;
21     bool producesMidi() const override;
22     bool isMidiEffect() const override;
23     double getTailLengthSeconds() const override;
24
25     int getNumPrograms() override;
26     int getCurrentProgram() override;
27     void setCurrentProgram(int index) override;
28     const String getProgramName(int index) override;
29     void changeProgramName(int index, const String& newName) override;
30
31     void getStateInformation(MemoryBlock& destData) override;
32     void setStateInformation(const void* data, int sizeInBytes) override;
33
34 private:
35     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(PluginProcessor)
36 };
```

Parameters via references only

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      PluginProcessor();
4      // ...
5      struct Parameters {
6          explicit Parameters(juce::AudioProcessor&);
7          juce::AudioParameterFloat& rate;
8          juce::AudioParameterBool& bypassed;
9          juce::AudioParameterChoice& waveform;
10     };
11     Parameters parameters{*this};
12 };
```

Parameters via references only

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      PluginProcessor();
4      // ...
5      struct Parameters {
6          explicit Parameters(juce::AudioProcessor&);
7          juce::AudioParameterFloat& rate;
8          juce::AudioParameterBool& bypassed;
9          juce::AudioParameterChoice& waveform;
10     };
11     Parameters parameters{*this};
12 };;
```

Parameters via references only

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      PluginProcessor();
4      // ...
5      struct Parameters {
6          explicit Parameters(juce::AudioProcessor&);
7          juce::AudioParameterFloat& rate;
8          juce::AudioParameterBool& bypassed;
9          juce::AudioParameterChoice& waveform;
10     };
11     Parameters parameters{*this};
12 };
```

Parameters via references only

```
1 namespace {
2     auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3         auto& result = *parameter;
4         processor.addParameter(parameter.release());
5         return result;
6     }
7
8     juce::AudioParameterFloat& createModulationRateParameter(
9         juce::AudioProcessor& processor) {
10         constexpr auto versionHint = 1;
11         return addParameterToProcessor(
12             processor,
13             std::make_unique<juce::AudioParameterFloat>(<
14                 juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15                 juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16                 juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
17     }
18
19     juce::AudioParameterBool& createBypassedParameter(
20         juce::AudioProcessor& processor) {
21         constexpr auto versionHint = 1;
22         return addParameterToProcessor(
23             processor,
24             std::make_unique<juce::AudioParameterBool>(<
25                 juce::ParameterID{"bypassed", versionHint}, "Bypass", false));
26     }
27
28     juce::AudioParameterChoice& createWaveformParameter(
29         juce::AudioProcessor& processor) {
30         constexpr auto versionHint = 1;
31         return addParameterToProcessor(
32             processor,
33             std::make_unique<juce::AudioParameterChoice>(<
34                 juce::ParameterID{"modulation.waveform", versionHint},
35                 "Modulation waveform", juce::StringArray{"Sine", "Triangle", 0});
36     }
37 } // namespace
38
39 Parameters::Parameters(juce::AudioProcessor& p)
40 : rate{createModulationRateParameter(p)},
41   bypassed{createBypassedParameter(p)},
42   waveform{createWaveformParameter(p)} {}
```

Parameters via references only

```
1 namespace {
2 auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3     auto& result = *parameter;
4     processor.addParameter(parameter.release());
5     return result;
6 }
7
8 juce::AudioParameterFloat& createModulationRateParameter(
9     juce::AudioProcessor& processor) {
10     constexpr auto versionHint = 1;
11     return addParameterToProcessor(
12         processor,
13         std::make_unique<juce::AudioParameterFloat>(<
14             juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15             juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16             juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
17 }
18 } // namespace
19
20 Parameters::Parameters(juce::AudioProcessor& p)
21     : rate{createModulationRateParameter(p)},
22     /* ... */ {}
```

Parameters via references only

```
1 namespace {
2 auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3     auto& result = *parameter;
4     processor.addParameter(parameter.release());
5     return result;
6 }
7
8 juce::AudioParameterFloat& createModulationRateParameter(
9     juce::AudioProcessor& processor) {
10     constexpr auto versionHint = 1;
11     return addParameterToProcessor(
12         processor,
13         std::make_unique<juce::AudioParameterFloat>(<
14             juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15             juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16             juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
17 }
18 } // namespace
19
20 Parameters::Parameters(juce::AudioProcessor& p)
21     : rate{createModulationRateParameter(p)},
22     /* ... */ {}
```

Parameters via references only

```
1 namespace {
2     auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3         auto& result = *parameter;
4         processor.addParameter(parameter.release());
5         return result;
6     }
7
8     juce::AudioParameterFloat& createModulationRateParameter(
9         juce::AudioProcessor& processor) {
10         constexpr auto versionHint = 1;
11         return addParameterToProcessor(
12             processor,
13             std::make_unique<juce::AudioParameterFloat>(<
14                 juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15                 juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16                 juce::AudioParameterFloatAttributes{}.withLabel("Hz"))));
17     }
18 } // namespace
19
20 Parameters::Parameters(juce::AudioProcessor& p)
21     : rate{createModulationRateParameter(p)},
22     /* ... */ {}
```

Parameters via references only

```
1 namespace {
2     auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3         auto& result = *parameter;
4         processor.addParameter(parameter.release());
5         return result;
6     }
7
8     juce::AudioParameterFloat& createModulationRateParameter(
9         juce::AudioProcessor& processor) {
10         constexpr auto versionHint = 1;
11         return addParameterToProcessor(
12             processor,
13             std::make_unique<juce::AudioParameterFloat>(<
14                 juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15                 juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16                 juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
17     }
18 } // namespace
19
20 Parameters::Parameters(juce::AudioProcessor& p)
21     : rate{createModulationRateParameter(p)},
22     /* ... */ {}
```

Parameters via references only

```
1 namespace {
2     auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3         auto& result = *parameter;
4         processor.addParameter(parameter.release());
5         return result;
6     }
7
8     juce::AudioParameterFloat& createModulationRateParameter(
9         juce::AudioProcessor& processor) {
10         constexpr auto versionHint = 1;
11         return addParameterToProcessor(
12             processor,
13             std::make_unique<juce::AudioParameterFloat>(<
14                 juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15                 juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16                 juce::AudioParameterFloatAttributes{}.withLabel("Hz"))));
17     }
18 } // namespace
19
20 Parameters::Parameters(juce::AudioProcessor& p)
21     : rate{createModulationRateParameter(p)},
22     /* ... */ {}
```

Parameters via references only

```
1 namespace {
2 auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
3     auto& result = *parameter;
4     processor.addParameter(parameter.release());
5     return result;
6 }
7
8 juce::AudioParameterFloat& createModulationRateParameter(
9     juce::AudioProcessor& processor) {
10     constexpr auto versionHint = 1;
11     return addParameterToProcessor(
12         processor,
13         std::make_unique<juce::AudioParameterFloat>(<
14             juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
15             juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
16             juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
17 }
18 } // namespace
19
20 Parameters::Parameters(juce::AudioProcessor& p)
21     : rate{createModulationRateParameter(p)},
22     /* ... */ {}
```

Parameters via references only

Usage in audio processing

```
1 void PluginProcessor::processBlock(juce::AudioBuffer<float>& buffer,  
2                                   juce::MidiBuffer&) {  
3     // ...  
4     tremolo.setModulationRateHz(parameters.rate.get());  
5     tremolo.setLfoWaveform(  
6         static_cast<Tremolo::LfoWaveform>(parameters.waveform.getIndex()));  
7     bypassTransitionSmoother.setBypass(parameters.bypassed.get());  
8  
9     // audio processing  
10 }
```

Parameters via references only

Usage in UI

```
1  class PluginEditor : public juce::AudioProcessorEditor {
2  public:
3      explicit PluginEditor(PluginProcessor&);
4
5      void resized() override;
6
7  private:
8      juce::ComboBox waveformComboBox;
9      juce::ComboBoxParameterAttachment waveformAttachment;
10
11     juce::Slider rateSlider;
12     juce::SliderParameterAttachment rateAttachment;
13
14     juce::ToggleButton bypassButton{"BYPASSED"};
15     juce::ButtonParameterAttachment bypassAttachment;
16 };
```

Parameters via references only

Usage in UI

```
1  class PluginEditor : public juce::AudioProcessorEditor {
2  public:
3      explicit PluginEditor(PluginProcessor&);
4
5      void resized() override;
6
7  private:
8      juce::ComboBox waveformComboBox;
9      juce::ComboBoxParameterAttachment waveformAttachment;
10
11     juce::Slider rateSlider;
12     juce::SliderParameterAttachment rateAttachment;
13
14     juce::ToggleButton bypassButton{"BYPASSED"};
15     juce::ButtonParameterAttachment bypassAttachment;
16 };
```

Parameters via references only

Usage in UI

```
1  class PluginEditor : public juce::AudioProcessorEditor {
2  public:
3      explicit PluginEditor(PluginProcessor&);
4
5      void resized() override;
6
7  private:
8      juce::ComboBox waveformComboBox;
9      juce::ComboBoxParameterAttachment waveformAttachment;
10
11     juce::Slider rateSlider;
12     juce::SliderParameterAttachment rateAttachment;
13
14     juce::ToggleButton bypassButton{"BYPASSED"};
15     juce::ButtonParameterAttachment bypassAttachment;
16 };
```

Parameters via references only

Usage in UI

```
1  PluginEditor::PluginEditor(PluginProcessor& p)
2      : AudioProcessorEditor(&p),
3        waveformAttachment{p.parameters.waveform, waveformComboBox},
4        rateAttachment{p.parameters.rate, rateSlider},
5        bypassAttachment{p.parameters.bypassed, bypassButton} {}
```

Parameters via references only

Usage in UI

```
1  PluginEditor::PluginEditor(PluginProcessor& p)
2      : AudioProcessorEditor(&p),
3        waveformAttachment{p.parameters.waveform, waveformComboBox},
4        rateAttachment{p.parameters.rate, rateSlider},
5        bypassAttachment{p.parameters.bypassed, bypassButton} {}
```

Parameters via references only

Serialization

```
1 void PluginProcessor::getStateInformation(juce::MemoryBlock& destData) {
2     juce::MemoryOutputStream outputStream{destData, true};
3     JsonSerializer::serialize(parameters, outputStream);
4 }
5
6 void PluginProcessor::setStateInformation(const void* data, int sizeInBytes) {
7     juce::MemoryInputStream inputStream{data, static_cast<size_t>(sizeInBytes),
8                                             false};
9     const auto result = JsonSerializer::deserialize(inputStream, parameters);
10    if (result.failed()) {
11        // notify the user
12    }
13    // optionally skip smoothing
14 }
```

Parameters via references only

Serialization

```
1 void PluginProcessor::getStateInformation(juce::MemoryBlock& destData) {
2     juce::MemoryOutputStream outputStream{destData, true};
3     JsonSerializer::serialize(parameters, outputStream);
4 }
5
6 void PluginProcessor::setStateInformation(const void* data, int sizeInBytes) {
7     juce::MemoryInputStream inputStream{data, static_cast<size_t>(sizeInBytes),
8                                             false};
9     const auto result = JsonSerializer::deserialize(inputStream, parameters);
10    if (result.failed()) {
11        // notify the user
12    }
13    // optionally skip smoothing
14 }
```

Parameters via references only

Serialization

```
1 void PluginProcessor::getStateInformation(juce::MemoryBlock& destData) {
2     juce::MemoryOutputStream outputStream{destData, true};
3     JsonSerializer::serialize(parameters, outputStream);
4 }
5
6 void PluginProcessor::setStateInformation(const void* data, int sizeInBytes) {
7     juce::MemoryInputStream inputStream{data, static_cast<size_t>(sizeInBytes),
8                                             false};
9     const auto result = JsonSerializer::deserialize(inputStream, parameters);
10    if (result.failed()) {
11        // notify the user
12    }
13    // optionally skip smoothing
14 }
```

Parameters via references only

Serialization

```
1 void PluginProcessor::getStateInformation(juce::MemoryBlock& destData) {
2     juce::MemoryOutputStream outputStream{destData, true};
3     JsonSerializer::serialize(parameters, outputStream);
4 }
5
6 void PluginProcessor::setStateInformation(const void* data, int sizeInBytes) {
7     juce::MemoryInputStream inputStream{data, static_cast<size_t>(sizeInBytes),
8                                             false};
9     const auto result = JsonSerializer::deserialize(inputStream, parameters);
10    if (result.failed()) {
11        // notify the user
12    }
13    // optionally skip smoothing
14 }
```

Parameters via references only

Serialization

```
1  struct SerializableParameters {
2      float rate;
3      bool bypassed;
4      juce::String waveform;
5
6      static constexpr auto marshallVersion = 1;
7
8      template <typename Archive, typename T>
9      static void serialise(Archive& archive, T& p) {
10         using namespace juce;
11
12         if (archive.getVersion() != 1) {
13             return;
14         }
15
16         std::string pluginName = TREMOLO_PLUGIN_NAME;
17
18         archive(named("pluginName", pluginName));
19
20         if (pluginName != TREMOLO_PLUGIN_NAME) {
21             return;
22         }
23
24         archive(named("modulationRateHz", p.rate), named("bypassed", p.bypassed),
25                 named("modulationWaveform", p.waveform));
26     }
27 };
```

Parameters via references only

Serialization

```
1  struct SerializableParameters {
2      float rate;
3      bool bypassed;
4      juce::String waveform;
5
6      static constexpr auto marshallVersion = 1;
7
8      template <typename Archive, typename T>
9      static void serialise(Archive& archive, T& p) {
10         using namespace juce;
11
12         if (archive.getVersion() != 1) {
13             return;
14         }
15
16         std::string pluginName = TREMOLO_PLUGIN_NAME;
17
18         archive(named("pluginName", pluginName));
19
20         if (pluginName != TREMOLO_PLUGIN_NAME) {
21             return;
22         }
23
24         archive(named("modulationRateHz", p.rate), named("bypassed", p.bypassed),
25                 named("modulationWaveform", p.waveform));
26     }
27 };
```

```
1  SerializableParameters from(const Parameters& p) {
2      return {
3          .rate = p.rate.get(),
4          .bypassed = p.bypassed.get(),
5          .waveform = p.waveform.getCurrentChoiceName(),
6      };
7  }
8
9  void JsonSerializer::serialize(const Parameters& parameters,
10                               juce::OutputStream& output) {
11      const auto json = juce::ToVar::convert(from(parameters));
12
13      if (!json.has_value()) {
14          return;
15      }
16
17      juce::JSON::writeToStream(output, *json,
18                               juce::JSON::FormatOptions{}
19                               .withSpacing(juce::JSON::Spacing::multiLine)
20                               .withMaxDecimalPlaces(2));
21  }
```

```
1  SerializableParameters from(const Parameters& p) {
2      return {
3          .rate = p.rate.get(),
4          .bypassed = p.bypassed.get(),
5          .waveform = p.waveform.getCurrentChoiceName(),
6      };
7  }
8
9  void JsonSerializer::serialize(const Parameters& parameters,
10                               juce::OutputStream& output) {
11      const auto json = juce::ToVar::convert(from(parameters));
12
13      if (!json.has_value()) {
14          return;
15      }
16
17      juce::JSON::writeToStream(output, *json,
18                               juce::JSON::FormatOptions{}
19                               .withSpacing(juce::JSON::Spacing::multiLine)
20                               .withMaxDecimalPlaces(2));
21  }
```

```
1  SerializableParameters from(const Parameters& p) {
2      return {
3          .rate = p.rate.get(),
4          .bypassed = p.bypassed.get(),
5          .waveform = p.waveform.getCurrentChoiceName(),
6      };
7  }
8
9  void JsonSerializer::serialize(const Parameters& parameters,
10                               juce::OutputStream& output) {
11      const auto json = juce::ToVar::convert(from(parameters));
12
13      if (!json.has_value()) {
14          return;
15      }
16
17      juce::JSON::writeToStream(output, *json,
18                               juce::JSON::FormatOptions{}
19                                   .withSpacing(juce::JSON::Spacing::multiLine)
20                                   .withMaxDecimalPlaces(2));
21  }
```

Parameters via references only

Parameters via references only

Pros

Parameters via references only

Pros

- Full type safety
- Easy access to singular parameters
 - `processBlock()`
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization

Parameters via references only

Pros

- Full type safety
- Easy access to singular parameters
 - `processBlock()`
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization

Cons

Parameters via references only

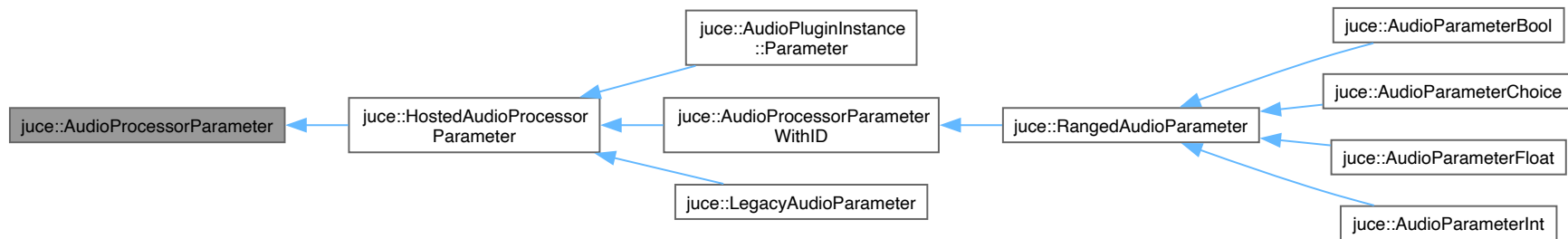
Pros

- Full type safety
- Easy access to singular parameters
 - `processBlock()`
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization

Cons

- "Manual" serialization code
 - Adding new parameters requires updating `JsonSerializer` $\implies$ error-prone

Parameters via a vector of base class pointers



```
1  class PluginProcessor : public juce::AudioProcessor {
2      // ...
3      std::vector<juce::RangedAudioParameter*> parameters;
4  };
```

Parameters via a vector of base class pointers

```
1  class JUCE_API AudioParameterBool : public RangedAudioParameter
2  {
3  public:
4      bool get() const noexcept;
5      operator bool() const noexcept;
6      // ...
7  };
8
9  class JUCE_API RangedAudioParameter : public AudioProcessorParameterWithID
10 {
11 public:
12     float convertTo0to1(float v) const noexcept;
13     float convertFrom0to1(float v) const noexcept;
14
15     // inherited:
16
17     /* Hosts will expect the value returned to be between 0 and 1.0. */
18     float getValue() const;
19
20     /* The value passed will be between 0 and 1.0. */
21     void setValue(float newValue);
22 };
```

Parameters via a vector of base class pointers

```
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <EdenSynthParameters>
4    <PARAM id="envelope.adbdr.attack.curve" value="1.0" />
5    <PARAM id="envelope.adbdr.attack.time" value="30.0" />
6    <PARAM id="envelope.adbdr.breakLevel" value="0.6000000238418579" />
7    <PARAM id="envelope.adbdr.decay1.curve" value="1.0" />
8    <PARAM id="envelope.adbdr.decay1.time" value="20.0" />
9    <PARAM id="envelope.adbdr.decay2.curve" value="1.0" />
10   <PARAM id="envelope.adbdr.decay2.time" value="20000.0" />
11   <PARAM id="envelope.adbdr.release.curve" value="1.0" />
12   <PARAM id="envelope.adbdr.release.time" value="300.0" />
13   <PARAM id="filter.contourAmount" value="1.0" />
14   <PARAM id="filter.cutoff" value="1.0" />
15   <PARAM id="filter.passbandAttenuation" value="0.0" />
16   <PARAM id="filter.resonance" value="0.0" />
17   <PARAM id="frequencyOfA4" value="440.0" />
18   <!-- more parameters ... -->
19   <PARAM id="output.volume" value="1.0" />
20   <PARAM id="pitchBend.semitonesDown" value="-12.0" />
21   <PARAM id="pitchBend.semitonesUp" value="2.0" />
22   <PARAM id="waveshaper.autoMakeUpGain" value="0.0" />
23 </EdenSynthParameters>
```

Summary so far

1. We can treat plugin parameters individually using only concrete

`juce::AudioParameterFloat|Bool|Int|Choice` classes $\implies$ We cannot (easily) define operations on a collection of parameters

2. We can treat plugin parameters as a collection of `juce::RangedAudioParameter` s $\implies$ We lose type information

Summary so far

1. Parameter collection: extensibility
2. Individual parameters: interpretability

How can we treat parameters as a collection without losing type information? 🤔

How can we treat parameters as a collection without losing type information? 🤔

Answer: Type Erasure!

Type-Erased Parameters

Collection of parameters

```
1  class TypeErasedParameter;  
2  
3  std::vector<TypeErasedParameter> parameters;
```

TypeErasedParameter

```
1 class TypeErasedParameter {  
2 public:  
3     TypeErasedParameter(juce::AudioParameterFloat& p) : _p{p} {}  
4  
5 private:  
6     juce::AudioParameterFloat& _p;  
7 };
```

TypeErasedParameter

```
1 class TypeErasedParameter {
2 public:
3     template <class Parameter>
4     TypeErasedParameter(Parameter& p) : _p{p} {}
5
6 private:
7     Parameter& _p;
8 };
```

TypeErasedParameter

```
1  template <class Parameter>
2  class TypeErasedParameter {
3  public:
4      TypeErasedParameter(Parameter& p) : _p{p} {}
5
6  private:
7      Parameter& _p;
8  };
```

TypeErasedParameter

```
1  template <class Parameter>
2  class TypeErasedParameter {
3  public:
4      TypeErasedParameter(Parameter& p) : _p{p} {}
5
6  private:
7      Parameter& _p;
8  };
9
10 std::vector<TypeErasedParameter<?>> parameters;
```

TypeErasedParameter

```
1  class TypeErasedParameter {
2  public:
3      template <class Parameter>
4      TypeErasedParameter(Parameter& p) : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
5
6  private:
7      template <class Parameter>
8      class ParameterModel {
9      public:
10         ParameterModel(Parameter& p) : _p{p} {}
11     private:
12         Parameter& _p;
13     };
14
15     std::unique_ptr<ParameterModel<?>> _impl; // ← problem
16 };
```

TypeErasedParameter

```
1  class TypeErasedParameter {
2  public:
3      template <class Parameter>
4      TypeErasedParameter(Parameter& p) : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
5
6  private:
7      class ParameterConcept {
8      public:
9          virtual ~ParameterConcept() = default;
10     };
11
12     template <class Parameter>
13     class ParameterModel : public ParameterConcept {
14     public:
15         ParameterModel(Parameter& p) : _p{p} {}
16     private:
17         Parameter& _p;
18     };
19
20     std::unique_ptr<ParameterConcept> _impl;
21 };
```

TypeErasedParameter

```
1  class TypeErasedParameter {
2  public:
3      template <class Parameter>
4      TypeErasedParameter(Parameter& p) : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
5
6  private:
7      class ParameterConcept {
8      public:
9          virtual ~ParameterConcept() = default;
10     };
11
12     template <class Parameter>
13     class ParameterModel : public ParameterConcept {
14     public:
15         ParameterModel(Parameter& p) : _p{p} {}
16     private:
17         Parameter& _p;
18     };
19
20     std::unique_ptr<ParameterConcept> _impl;
21 };
22
23 std::vector<TypeErasedParameter> parameters;
```

Operations

```
1 void foo(juce::AudioParameterFloat& p);
2 void foo(juce::AudioParameterBool& p);
3 // ...
4 class TypeErasedParameter {
5 public:
6     // ...
7
8 private:
9     class ParameterConcept {
10 public:
11     virtual ~ParameterConcept() = default;
12 };
13
14 template <class Parameter>
15 class ParameterModel : public ParameterConcept {
16 public:
17     // ...
18 private:
19     Parameter& _p;
20 };
21
22 std::unique_ptr<ParameterConcept> _impl;
23 };
```

Operations

```
1 void foo(juce::AudioParameterFloat& p);
2 void foo(juce::AudioParameterBool& p);
3 // ...
4 class TypeErasedParameter {
5 public:
6     // ...
7
8 private:
9     class ParameterConcept {
10 public:
11     virtual ~ParameterConcept() = default;
12 };
13
14 template <class Parameter>
15 class ParameterModel : public ParameterConcept {
16 public:
17     // ...
18 private:
19     Parameter& _p;
20 };
21
22 std::unique_ptr<ParameterConcept> _impl;
23 };
```

Operations

```
1 void foo(juce::AudioParameterFloat& p);
2 void foo(juce::AudioParameterBool& p);
3 // ...
4 class TypeErasedParameter {
5 public:
6     // ...
7     void foo() { _impl->foo(); }
8 private:
9     class ParameterConcept {
10 public:
11     virtual ~ParameterConcept() = default;
12     virtual void foo() = 0;
13 };
14 template <class Parameter>
15 class ParameterModel : public ParameterConcept {
16 public:
17     // ...
18     void foo() override { foo(_p); }
19
20 private:
21     Parameter& _p;
22 };
23 std::unique_ptr<ParameterConcept> _impl;
24 };
```

Operations

```
1 void foo(juce::AudioParameterFloat& p);
2 void foo(juce::AudioParameterBool& p);
3 // ...
4 class TypeErasedParameter {
5 public:
6     // ...
7     void foo() { _impl->foo(); }
8 private:
9     class ParameterConcept {
10 public:
11     virtual ~ParameterConcept() = default;
12     virtual void foo() = 0;
13 };
14 template <class Parameter>
15 class ParameterModel : public ParameterConcept {
16 public:
17     // ...
18     void foo() override { foo(_p); }
19
20 private:
21     Parameter& _p;
22 };
23 std::unique_ptr<ParameterConcept> _impl;
24 };
```

Operations

```
1 void toJson(juce::AudioParameterFloat& p);
2 void toJson(juce::AudioParameterBool& p);
3 // ...
4 class TypeErasedParameter {
5 public:
6     // ...
7     void toJson() { _impl->toJson(); }
8 private:
9     class ParameterConcept {
10 public:
11     virtual ~ParameterConcept() = default;
12     virtual void toJson() = 0;
13 };
14 template <class Parameter>
15 class ParameterModel : public ParameterConcept {
16 public:
17     // ...
18     void toJson() override { toJson(_p); }
19 private:
20     Parameter& _p;
21 };
22 std::unique_ptr<ParameterConcept> _impl;
23 };
24 }
```

Serialization

```
1  struct Serializer {
2      virtual ~Serializer = default;
3      virtual void serializeToJson(juce::AudioParameterFloat& p) = 0;
4      virtual void serializeToJson(juce::AudioParameterBool& p) = 0;
5      // ...
6  };
7
8  class TypeErasedParameter {
9  public:
10     // ...
11     void serialize(Serializer& s) { _impl->serialize(s); }
12 private:
13     class ParameterConcept {
14     public:
15         virtual ~ParameterConcept() = default;
16         virtual void serialize(Serializer&) = 0;
17     };
18     template <class Parameter>
19     class ParameterModel : public ParameterConcept {
20     public:
21         // ...
22         void serialize(Serializer& s) override { serializer.serialize(_p); }
23
24     private:
```

Operations supporting all JUCE parameter classes

```
1  struct Visitor {
2      virtual ~Visitor = default;
3      virtual void visit(juce::AudioParameterFloat& p) = 0;
4      virtual void visit(juce::AudioParameterBool& p) = 0;
5      // ...
6  };
7
8  class TypeErasedParameter {
9  public:
10     // ...
11     void accept(Visitor& v) { _impl->accept(v); }
12 private:
13     class ParameterConcept {
14     public:
15         virtual ~ParameterConcept() = default;
16         virtual void accept(Visitor&) = 0;
17     };
18     template <class Parameter>
19     class ParameterModel : public ParameterConcept {
20     public:
21         // ...
22         void accept(Visitor& v) override { v.visit(_p); }
23
24     private:
```

Collection of TypeErasedParameters

```
1 std::vector<TypeErasedParameter> parameters;
```

Collection of TypeErasedParameters

```
1  class ParameterHolder {
2      class TypeErasedParameter {
3      public:
4          template <class Parameter>
5          explicit TypeErasedParameter(Parameter& p)
6              : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7
8          void accept(Visitor& v) { _impl->accept(v); }
9
10     private:
11         class ParameterConcept { // NOLINT
12         public:
13             virtual ~ParameterConcept() = default;
14             virtual void accept(Visitor& v) = 0;
15         };
16
17         template <class Parameter>
18         class ParameterModel : public ParameterConcept {
19         public:
20             explicit ParameterModel(Parameter& p) : _p{p} {}
21
22             void accept(Visitor& v) override { v.visit(_p.get()); }
23
24         private:
25             Parameter& _p;
26         };
27
28         std::unique_ptr<ParameterConcept> _impl;
29     };
30
31     public:
32     explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
33         : _parameters{std::move(parameters)} {}
34
35     void accept(Visitor& v) {
36         for (auto& parameter : _parameters) {
37             parameter.accept(v);
38         }
39     }
40
41     private:
42     std::vector<TypeErasedParameter> _parameters;
43 };
```

Collection of TypeErasedParameters

```
1 class ParameterHolder {
2     class TypeErasedParameter {
3     public:
4         template <class Parameter>
5         explicit TypeErasedParameter(Parameter& p)
6             : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7
8         void accept(Visitor& v) { _impl->accept(v); }
9
10    private:
11        class ParameterConcept { // NOLINT
12        public:
13            virtual ~ParameterConcept() = default;
14            virtual void accept(Visitor& v) = 0;
15        };
16
17        template <class Parameter>
18        class ParameterModel : public ParameterConcept {
19        public:
20            explicit ParameterModel(Parameter& p) : _p{p} {}
21
22            void accept(Visitor& v) override { v.visit(_p.get()); }
23
24        private:
25            Parameter& _p;
26        };
27
28        std::unique_ptr<ParameterConcept> _impl;
29    };
30
31    public:
32        explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
33            : _parameters{std::move(parameters)} {}
34
35        void accept(Visitor& v) {
36            for (auto& parameter : _parameters) {
37                parameter.accept(v);
38            }
39        }
40
41    private:
42        std::vector<TypeErasedParameter> _parameters;
43    };
```

Collection of TypeErasedParameters

```
1 class ParameterHolder {
2     class TypeErasedParameter {
3     public:
4         template <class Parameter>
5         explicit TypeErasedParameter(Parameter& p)
6             : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7
8         void accept(Visitor& v) { _impl->accept(v); }
9
10    private:
11        class ParameterConcept { // NOLINT
12        public:
13            virtual ~ParameterConcept() = default;
14            virtual void accept(Visitor& v) = 0;
15        };
16
17        template <class Parameter>
18        class ParameterModel : public ParameterConcept {
19        public:
20            explicit ParameterModel(Parameter& p) : _p{p} {}
21
22            void accept(Visitor& v) override { v.visit(_p.get()); }
23
24        private:
25            Parameter& _p;
26        };
27
28        std::unique_ptr<ParameterConcept> _impl;
29    };
30
31    public:
32        explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
33            : _parameters{std::move(parameters)} {}
34
35        void accept(Visitor& v) {
36            for (auto& parameter : _parameters) {
37                parameter.accept(v);
38            }
39        }
40
41    private:
42        std::vector<TypeErasedParameter> _parameters;
43    };
```

Collection of TypeErasedParameters

```
1 class ParameterHolder {
2     class TypeErasedParameter {
3     public:
4         template <class Parameter>
5         explicit TypeErasedParameter(Parameter& p)
6             : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7
8         void accept(Visitor& v) { _impl->accept(v); }
9
10    private:
11        class ParameterConcept { // NOLINT
12        public:
13            virtual ~ParameterConcept() = default;
14            virtual void accept(Visitor& v) = 0;
15        };
16
17        template <class Parameter>
18        class ParameterModel : public ParameterConcept {
19        public:
20            explicit ParameterModel(Parameter& p) : _p{p} {}
21
22            void accept(Visitor& v) override { v.visit(_p.get()); }
23
24        private:
25            Parameter& _p;
26        };
27
28        std::unique_ptr<ParameterConcept> _impl;
29    };
30
31    public:
32        explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
33            : _parameters{std::move(parameters)} {}
34
35        void accept(Visitor& v) {
36            for (auto& parameter : _parameters) {
37                parameter.accept(v);
38            }
39        }
40
41    private:
42        std::vector<TypeErasedParameter> _parameters;
43    };
```

Collection of TypeErasedParameters

```
1 class ParameterHolder {
2     class TypeErasedParameter {
3     public:
4         template <class Parameter>
5         explicit TypeErasedParameter(Parameter& p)
6             : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7
8         void accept(Visitor& v) { _impl->accept(v); }
9
10    private:
11        class ParameterConcept { // NOLINT
12        public:
13            virtual ~ParameterConcept() = default;
14            virtual void accept(Visitor& v) = 0;
15        };
16
17        template <class Parameter>
18        class ParameterModel : public ParameterConcept {
19        public:
20            explicit ParameterModel(Parameter& p) : _p{p} {}
21
22            void accept(Visitor& v) override { v.visit(_p.get()); }
23
24        private:
25            Parameter& _p;
26        };
27
28        std::unique_ptr<ParameterConcept> _impl;
29    };
30
31    public:
32        explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
33            : _parameters{std::move(parameters)} {}
34
35        void accept(Visitor& v) {
36            for (auto& parameter : _parameters) {
37                parameter.accept(v);
38            }
39        }
40
41    private:
42        std::vector<TypeErasedParameter> _parameters;
43    };
```

Collection of TypeErasedParameters

```
1  class ParameterHolder {
2      class TypeErasedParameter {
3      public:
4          template <class Parameter>
5          explicit TypeErasedParameter(Parameter& p)
6              : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7
8          void accept(Visitor& v) { _impl->accept(v); }
9
10     private:
11         class ParameterConcept { // NOLINT
12         public:
13             virtual ~ParameterConcept() = default;
14             virtual void accept(Visitor& v) = 0;
15         };
16
17         template <class Parameter>
18         class ParameterModel : public ParameterConcept {
19         public:
20             explicit ParameterModel(Parameter& p) : _p{p} {}
21
22             void accept(Visitor& v) override { v.visit(_p.get()); }
23
24         private:
25             Parameter& _p;
26         };
27
28         std::unique_ptr<ParameterConcept> _impl;
29     };
30
31     public:
32     explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
33         : _parameters{std::move(parameters)} {}
34
35     void accept(Visitor& v) {
36         for (auto& parameter : _parameters) {
37             parameter.accept(v);
38         }
39     }
40
41     private:
42     std::vector<TypeErasedParameter> _parameters;
43 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class ... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class ... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

Builder

```
1  class Builder {
2  public:
3      template <class P, class ... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parametersForHolder.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     ParameterHolder build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return ParameterHolder{std::move(_parametersForHolder)};
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parametersForHolder;
22 };
```

```
1  class ParameterHolder {
2      class TypeErasedParameter {
3      public:
4          template <class Parameter>
5          explicit TypeErasedParameter(Parameter& p)
6              : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7          // ...
8      };
9
10 public:
11     class Builder {
12     public:
13         template <class P, class... Args>
14         P& add(Args&&... args) {
15             auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
16             auto& ref = *parameter;
17             _parametersForHolder.emplace_back(ref);
18             _parameters.push_back(std::move(parameter));
19             return ref;
20         }
21
22         ParameterHolder build(juce::AudioProcessor& p) && {
23             for (auto&& parameter : _parameters) {
24                 p.addParameter(parameter.release());
25             }
26             return ParameterHolder{std::move(_parametersForHolder)};
27         }
28
29     private:
30         std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
31         std::vector<TypeErasedParameter> _parametersForHolder;
32     };
33
34     void accept(Visitor& v) { /* ... */ }
35
36 private:
37     explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
38         : _parameters{std::move(parameters)} {}
39
40     std::vector<TypeErasedParameter> _parameters;
41 };
```

```
1  class ParameterHolder {
2      class TypeErasedParameter {
3      public:
4          template <class Parameter>
5          explicit TypeErasedParameter(Parameter& p)
6              : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
7          // ...
8      };
9
10     public:
11         class Builder {
12         public:
13             template <class P, class... Args>
14             P& add(Args&&... args) {
15                 auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
16                 auto& ref = *parameter;
17                 _parametersForHolder.emplace_back(ref);
18                 _parameters.push_back(std::move(parameter));
19                 return ref;
20             }
21
22             ParameterHolder build(juce::AudioProcessor& p) && {
23                 for (auto&& parameter : _parameters) {
24                     p.addParameter(parameter.release());
25                 }
26                 return ParameterHolder{std::move(_parametersForHolder)};
27             }
28
29         private:
30             std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
31             std::vector<TypeErasedParameter> _parametersForHolder;
32         };
33
34         void accept(Visitor& v) { /* ... */ }
35
36     private:
37         explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
38             : _parameters{std::move(parameters)} {}
39
40         std::vector<TypeErasedParameter> _parameters;
41     };
```

Usage

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      explicit PluginProcessor(
4          ParameterHolder::Builder builder = {})
5          : floatParam{builder.add<juce::AudioParameterFloat>(
6              "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f)},
7            boolParam{builder.add<juce::AudioParameterBool>(
8              "boolParam", "Bool Param", true)},
9            intParam{builder.add<juce::AudioParameterInt>(
10             "intParam", "Int Param", 5, 10, 6)},
11            choiceParam{builder.add<juce::AudioParameterChoice>(
12              "choiceParam", "Choice Param", juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)},
13            parameterHolder{std::move(builder).build(*this)} {}
14
15      // ...
16  private:
17      juce::AudioParameterFloat& floatParam;
18      juce::AudioParameterBool& boolParam;
19      juce::AudioParameterInt& intParam;
20      juce::AudioParameterChoice& choiceParam;
21      ParameterHolder parameterHolder;
22  };
```

Usage

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      explicit PluginProcessor(
4          ParameterHolder::Builder builder = {})
5          : floatParam{builder.add<juce::AudioParameterFloat>(
6              "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f)},
7            boolParam{builder.add<juce::AudioParameterBool>(
8              "boolParam", "Bool Param", true)},
9            intParam{builder.add<juce::AudioParameterInt>(
10             "intParam", "Int Param", 5, 10, 6)},
11            choiceParam{builder.add<juce::AudioParameterChoice>(
12              "choiceParam", "Choice Param", juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)},
13            parameterHolder{std::move(builder).build(*this)} {}
14
15     // ...
16 private:
17     juce::AudioParameterFloat& floatParam;
18     juce::AudioParameterBool& boolParam;
19     juce::AudioParameterInt& intParam;
20     juce::AudioParameterChoice& choiceParam;
21     ParameterHolder parameterHolder;
22 };
```

Usage

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      explicit PluginProcessor(
4          ParameterHolder::Builder builder = {})
5          : floatParam{builder.add<juce::AudioParameterFloat>(
6              "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f)},
7            boolParam{builder.add<juce::AudioParameterBool>(
8              "boolParam", "Bool Param", true)},
9            intParam{builder.add<juce::AudioParameterInt>(
10             "intParam", "Int Param", 5, 10, 6)},
11            choiceParam{builder.add<juce::AudioParameterChoice>(
12              "choiceParam", "Choice Param", juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)},
13            parameterHolder{std::move(builder).build(*this)} {}
14
15     // ...
16 private:
17     juce::AudioParameterFloat& floatParam;
18     juce::AudioParameterBool& boolParam;
19     juce::AudioParameterInt& intParam;
20     juce::AudioParameterChoice& choiceParam;
21     ParameterHolder parameterHolder;
22 };
```

Usage

```
1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      explicit PluginProcessor(
4          ParameterHolder::Builder builder = {})
5          : floatParam{builder.add<juce::AudioParameterFloat>(
6              "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f)},
7            boolParam{builder.add<juce::AudioParameterBool>(
8              "boolParam", "Bool Param", true)},
9            intParam{builder.add<juce::AudioParameterInt>(
10             "intParam", "Int Param", 5, 10, 6)},
11            choiceParam{builder.add<juce::AudioParameterChoice>(
12              "choiceParam", "Choice Param", juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)},
13            parameterHolder{std::move(builder).build(*this)} {}
14
15      // ...
16  private:
17      juce::AudioParameterFloat& floatParam;
18      juce::AudioParameterBool& boolParam;
19      juce::AudioParameterInt& intParam;
20      juce::AudioParameterChoice& choiceParam;
21      ParameterHolder parameterHolder;
22  };
```

Serialization using a Visitor

```
1  struct ParameterIdAndValue {
2      std::string id;
3      std::variant<float, int, bool, std::string> value;
4  };
5
6  class ParameterValuesExtractor : public Visitor {
7  public:
8      ParameterValuesExtractor() = default;
9      void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
10     void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
11     void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
12     void visit(juce::AudioParameterChoice& parameter) override {
13         visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
14     }
15     [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
16 private:
17     template <class P, class V>
18     void visitImpl(const P& parameter, V&& value) {
19         _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
20     }
21
22     std::vector<ParameterIdAndValue> _result;
23 };
```

Serialization using a Visitor

```
1  struct ParameterIdAndValue {
2      std::string id;
3      std::variant<float, int, bool, std::string> value;
4  };
5
6  class ParameterValuesExtractor : public Visitor {
7  public:
8      ParameterValuesExtractor() = default;
9      void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
10     void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
11     void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
12     void visit(juce::AudioParameterChoice& parameter) override {
13         visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
14     }
15     [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
16 private:
17     template <class P, class V>
18     void visitImpl(const P& parameter, V&& value) {
19         _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
20     }
21
22     std::vector<ParameterIdAndValue> _result;
23 };
```

Serialization using a Visitor

```
1  struct ParameterIdAndValue {
2      std::string id;
3      std::variant<float, int, bool, std::string> value;
4  };
5
6  class ParameterValuesExtractor : public Visitor {
7  public:
8      ParameterValuesExtractor() = default;
9      void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
10     void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
11     void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
12     void visit(juce::AudioParameterChoice& parameter) override {
13         visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
14     }
15     [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
16 private:
17     template <class P, class V>
18     void visitImpl(const P& parameter, V&& value) {
19         _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
20     }
21
22     std::vector<ParameterIdAndValue> _result;
23 };
```

Serialization using a Visitor

```
1  struct ParameterIdAndValue {
2      std::string id;
3      std::variant<float, int, bool, std::string> value;
4  };
5
6  class ParameterValuesExtractor : public Visitor {
7  public:
8      ParameterValuesExtractor() = default;
9      void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
10     void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
11     void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
12     void visit(juce::AudioParameterChoice& parameter) override {
13         visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
14     }
15     [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
16 private:
17     template <class P, class V>
18     void visitImpl(const P& parameter, V&& value) {
19         _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
20     }
21
22     std::vector<ParameterIdAndValue> _result;
23 };
```

Serialization using a Visitor

```
1  struct ParameterIdAndValue {
2      std::string id;
3      std::variant<float, int, bool, std::string> value;
4  };
5
6  class ParameterValuesExtractor : public Visitor {
7  public:
8      ParameterValuesExtractor() = default;
9      void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
10     void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
11     void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
12     void visit(juce::AudioParameterChoice& parameter) override {
13         visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
14     }
15     [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
16 private:
17     template <class P, class V>
18     void visitImpl(const P& parameter, V&& value) {
19         _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
20     }
21
22     std::vector<ParameterIdAndValue> _result;
23 };
```

Serialization using a Visitor

```
1  struct ParameterIdAndValue {
2      std::string id;
3      std::variant<float, int, bool, std::string> value;
4  };
5
6  class ParameterValuesExtractor : public Visitor {
7  public:
8      ParameterValuesExtractor() = default;
9      void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
10     void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
11     void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
12     void visit(juce::AudioParameterChoice& parameter) override {
13         visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
14     }
15     [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
16 private:
17     template <class P, class V>
18     void visitImpl(const P& parameter, V&& value) {
19         _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
20     }
21
22     std::vector<ParameterIdAndValue> _result;
23 };
```

Serialization using a Visitor

```
1  std::vector<ParameterIdAndValue> parameterIdsAndValues(ParameterHolder& ph) {  
2      ParameterValuesExtractor visitor;  
3      ph.accept(visitor);  
4      return visitor.result();  
5  }
```

What if we want to support custom parameter classes?

⇒ make `ParameterHolder` templated on the `Visitor` class.

What if we want to support custom parameter classes?

```
1  template <class Visitor>
2  class ParameterHolder {
3      // ...
4  public:
5      void accept(Visitor& v) { /* ... */ }
6      // ...
7  };
```

What if we want to support custom parameter classes?

A good default

```
1  struct JuceParameterVisitor {
2      virtual ~JuceParameterVisitor() = default;
3      virtual void visit(juce::AudioParameterBool&) = 0;
4      virtual void visit(juce::AudioParameterFloat&) = 0;
5      virtual void visit(juce::AudioParameterInt&) = 0;
6      virtual void visit(juce::AudioParameterChoice&) = 0;
7  };
8
9  using JuceParameterHolder = ParameterHolder<JuceParameterVisitor>;
```

Implementations

ParameterHolder with example serialization

- <https://github.com/JanWilczek/wolfsound-dsp-utils>
 - *src/include/wolfsound/juce/wolfsound_ParameterHolder.hpp*

ParameterHolder with serialization and presets (WIP)

- <https://github.com/JanWilczek/EdenSynth/tree/add-xml-presets-macos-var-params>
 - *EdenSynth/SharedCode/include/presets/Preset.h*
 - *EdenSynth/SharedCode_test/source/presets_test/PresetsTest.cpp*

Conclusion

A good use of Type Erasure is to hold a collection of strongly-typed objects (that may or may not share a base class), when the types of the objects are not fixed (or we have no control over those types) and we want to perform common actions for all elements of the collection.

Another example: `tc::any_range_ref`

```
1  template <typename T>
2  struct any_range_ref {
3      template <typename Rng>
4      any_range_ref(Rng&& rng) noexcept
5          : m_pfuncTypeErased(
6              [](tc::no_adl::any_ref anyrefRng,
7                 tc::no_adl::function_ref<tc::break_or_continue (T) noexcept> fn) noexcept → tc::break_or_continue {
8                  return tc::for_each(anyrefRng.get_ref<std::remove_reference_t<Rng>>(), fn);
9              }
10         )
11         , m_anyrefRng(tc::as_lvalue(rng))
12     {}
13
14     tc::break_or_continue
15     operator()(tc::no_adl::function_ref<tc::break_or_continue (T) noexcept> fn) const& noexcept {
16         return m_pfuncTypeErased(m_anyrefRng, fn);
17     }
18
19 private:
20     tc::no_adl::type_erased_function_ptr<
21         /*bNoExcept*/true, tc::break_or_continue, tc::no_adl::function_ref<tc::break_or_continue (T) noexcept>>
22         m_pfuncTypeErased;
23     tc::no_adl::any_ref m_anyrefRng;
24 };
```

Another example: `tc::any_range_ref`

```
1  auto stringify_concat(tc::any_range_ref<int> anyrngref) {
2      return tc::make<tc::string>(tc::join_with_separator(",", " ", tc::transform(anyrngref, tc_fn(tc::as_dec))));
3  }
4
5  assert("0, 1, 2, 3, 4, 5" == (stringify_concat(std::vector<int>{0, 1, 2, 3, 4, 5})));
6  assert("0, 1, 2, 3, 4, 5" == (stringify_concat(std::list<int>{0, 1, 2, 3, 4, 5})));
7
8  // TODO: vector of any_range_ref?
```

Another example: `edit_combobox`

References

References

1. JUCE C++ framework source code, <https://github.com/juce-framework/JUCE>

References

1. JUCE C++ framework source code, <https://github.com/juce-framework/JUCE>
2. Kevlin Henney, *Valued Conversions*, *C++ Report* July-August 2000

References

1. JUCE C++ framework source code, <https://github.com/juce-framework/JUCE>
2. Kevlin Henney, *Valued Conversions*, *C++ Report* July-August 2000
3. Sean Parent, *Inheritance Is the Base Class of Evil*, *GoingNative* 2013

References

1. JUCE C++ framework source code, <https://github.com/juce-framework/JUCE>
2. Kevlin Henney, *Valued Conversions*, *C++ Report* July-August 2000
3. Sean Parent, *Inheritance Is the Base Class of Evil*, GoingNative 2013
4. Klaus Iglberger, *C++ Software Design: Design Principles and Patterns for High-Quality Software*, O'Reilly 2022

References

1. JUCE C++ framework source code, <https://github.com/juce-framework/JUCE>
2. Kevlin Henney, *Valued Conversions*, *C++ Report* July-August 2000
3. Sean Parent, *Inheritance Is the Base Class of Evil*, GoingNative 2013
4. Klaus Iglberger, *C++ Software Design: Design Principles and Patterns for High-Quality Software*, O'Reilly 2022
5. Jan Wilczek & the JUCE team, *Official JUCE Audio Plugin Development Online Course*, <https://wolfsoundacademy.com/juce> (available for free)

Summary

Summary

1. General: Use the Type Erasure design pattern together with the Visitor design pattern to manage collections of strongly typed objects of different classes

Summary

1. General: Use the Type Erasure design pattern together with the Visitor design pattern to manage collections of strongly typed objects of different classes
2. Specific: Use Type Erasure to perform operations on all parameter objects of your plugin without losing their type
 - Keep references to concrete parameter objects to access them individually

Summary

1. General: Use the Type Erasure design pattern together with the Visitor design pattern to manage collections of strongly typed objects of different classes
2. Specific: Use Type Erasure to perform operations on all parameter objects of your plugin without losing their type
 - Keep references to concrete parameter objects to access them individually
3. Try out/tweak `wolfound::ParameterHolder` from github.com/JanWilczek/wolfound-dsp-utils

Summary

1. General: Use the Type Erasure design pattern together with the Visitor design pattern to manage collections of strongly typed objects of different classes
2. Specific: Use Type Erasure to perform operations on all parameter objects of your plugin without losing their type
 - Keep references to concrete parameter objects to access them individually
3. Try out/tweak `wolfound::ParameterHolder` from github.com/JanWilczek/wolfound-dsp-utils
4. Get slides at github.com/JanWilczek/ad26-japan-talk

Summary

1. General: Use the Type Erasure design pattern together with the Visitor design pattern to manage collections of strongly typed objects of different classes
2. Specific: Use Type Erasure to perform operations on all parameter objects of your plugin without losing their type
 - Keep references to concrete parameter objects to access them individually
3. Try out/tweak `wolfound::ParameterHolder` from github.com/JanWilczek/wolfound-dsp-utils
4. Get slides at github.com/JanWilczek/ad26-japan-talk
5. Contact me via contact@thewolfound.com